pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Docker: One Image Everywhere

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: one image everywhere

Today

1. What an image is
2. The Dockerfile, line by line
3. .dockerignore: what never goes in
4. check_dockerfile
5. Docker Desktop on Windows
6. Build, run, request, stop
7. Where an image could go public
8. One image everywhere

Where Docker runs

The notebook writes and checks

the files; it never runs Docker.

Docker's own output is shown as

masked transcripts, recorded once.

You run Docker in PowerShell,

on your own machine.

One file holds Linux, Python, the packages, and your code. It runs the same everywhere.

What the service returns

Day 6's bond service, unchanged: in a container it returns what it returned on your laptop.

Allowed names

price per 100 of face

DV01 per 100 of face

the service returned

Never called

fair value, cheap, or rich

a quote or a mark

a recommendation

An image changes where the service runs, never what it may say.

How an image works

The problem	A laptop and a server run two different stacks of OS, Python, and packages; the difference shows up as a wrong number
The method	The whole stack packed into one file, in layers: Linux, Python, the packages, your code, each from one Dockerfile line
A container	That image running: a process on the host that sees only the image's files; on Windows, inside a small Linux VM
You control	The base tag, the packages, the files copied, and the command that starts the server
What comes out	An image with a name and a tag, and a container that answers on a port you publish

What an image is

Your code	app.py, bondmath.py, deploykit.py
The packages	FastAPI, uvicorn, and what they need
Python	3.13.16, at an exact tag
An operating system	a small Debian Linux

Where it runs

This notebook

Windows, Python 3.13.8

The image, on every machine

Linux (Debian 13, trixie), Python 3.13.16, from python:3.13.16-slim

A container is the image running. On Windows, Docker Desktop runs it in a small Linux VM.

How a Dockerfile works

The problem	An image has to be built the same way every time, so the recipe is a file, not a memory
The method	docker build runs each instruction on the result of the one before: FROM, COPY, RUN (bash inside the image), CMD
The cache	Each result is a layer, reused while its line and everything above it are unchanged: requirements before code
You control	The base tag, the order of the lines, and the command
What comes out	The Dockerfile's text, to check here and build in PowerShell

The Dockerfile, line by line

```
FROM python:3.13.16-slim
WORKDIR /app
COPY requirements.txt .
RUN python -m pip install --no-cache-dir -r requirements.txt
COPY app.py bondmath.py deploykit.py ./
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["sh", "-c", "python -m uvicorn app:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

Read it aloud

exact tag, never latest

requirements first: cached

files copied by name

a user that is not root

RUN and CMD are bash

PORT, else 8000

--host 0.0.0.0: inside a container, 127.0.0.1 is the container alone.

How .dockerignore works

The problem	docker build . hands Docker the whole folder; any COPY line can take any file in it, a .env included
The method	A list of patterns read before the context is sent: a matching file never reaches Docker, whatever the Dockerfile says
Exceptions	A line starting with ! lets a named file back in: .env.example
You control	The patterns
What comes out	The .dockerignore text, written beside the Dockerfile and committed with it

What never goes in

26

files in the work folder, as COPY . . would take them

.env

among them: a key in every copy of the image, for good

Two guards

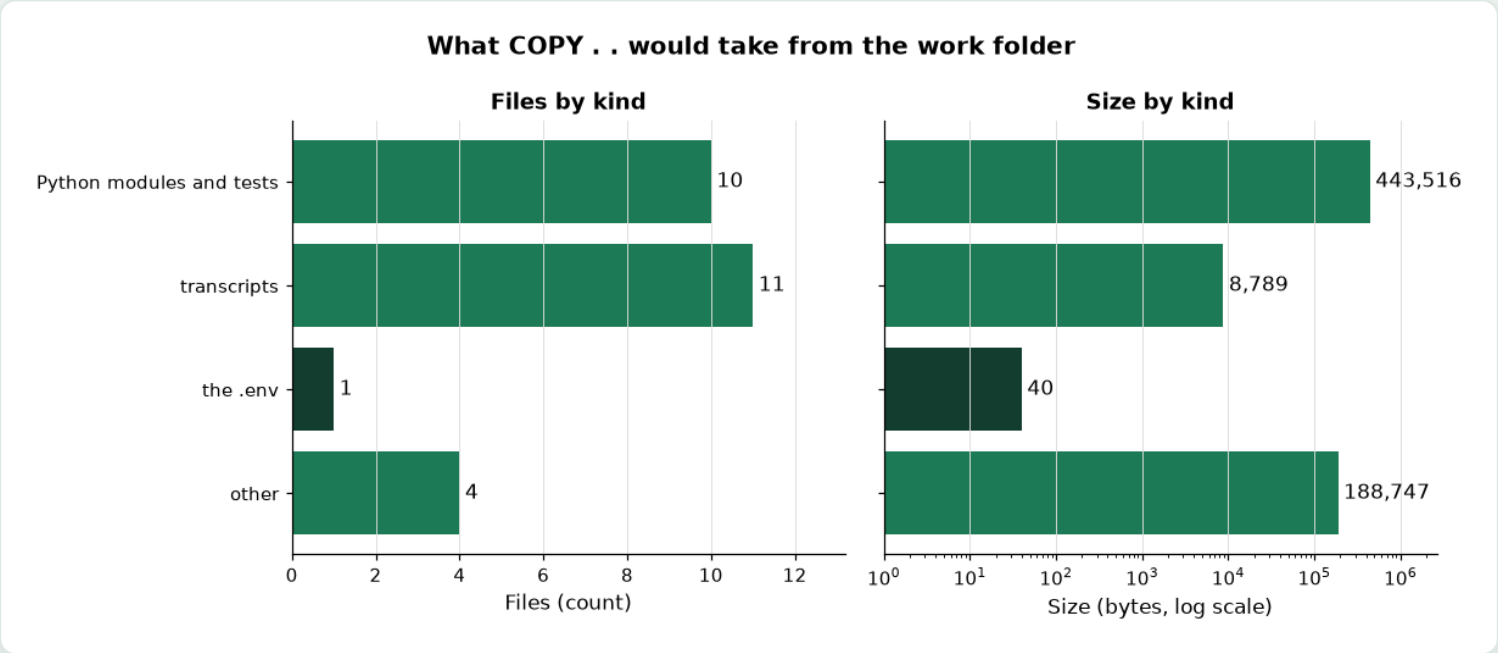
1. COPY names its files: .env never goes in.

2. .dockerignore keeps it out of the build context.

```
.env
.env.*
!.env.example
.git
cache/
__pycache__/
*.db
*.log
*.xlsx
.pytest_cache/
```

Deleting a .env and building again never reaches the copies already sent.

What COPY . . would take



Kind of file	Files	Bytes
Python modules and tests	10	443,516
transcripts	11	8,789
the .env	1	40
other	4	188,747

The .env is the dark bar: a few dozen bytes among the modules, and in every copy.

How check_dockerfile works

The problem	Docker checks a Dockerfile's grammar and nothing else: five mistakes build and run without a word
The method	FROM carries a tag, not latest; no secret's value in ENV or ARG; COPY . needs a .dockerignore for .env; a USER line; CMD in exec form
You control	The two texts
What comes out	A list of sentences naming the line and what to do, empty when the pair is safe to build

check_dockerfile: five problems that build and run

```
the tag latest: FROM python:latest: pin the image to a tag, never latest or none
a key in ENV: ENV HF_TOKEN: a secret's value in the image; set it on the platform
COPY . . with no .dockerignore: COPY . .: copies the whole folder, and .dockerignore does not exclude .env files
no USER line: no USER line: the server runs as root inside the container
CMD in shell form: CMD python -m uvicorn app:app --host 0.0.0.0 --port 8000: CMD in shell form; write it in exec fo...
```

0

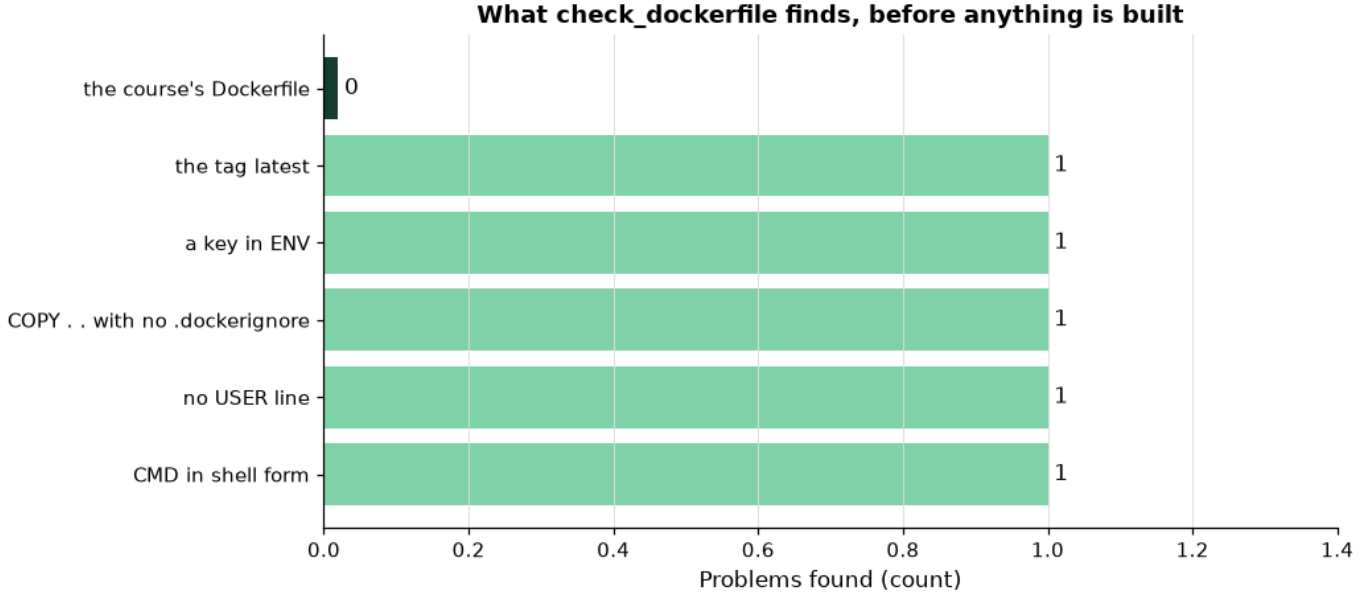
problems in the course's Dockerfile and .dockerignore

51

tests passed, four of them today's

Docker builds all five without a word. The check names each one before anything is built.

What the check finds



Dockerfile	Problems
the course's Dockerfile	0
the tag latest	1
a key in ENV	1
COPY . . with no .dockerignore	1
no USER line	1
CMD in shell form	1

Each planted Dockerfile builds and runs; the check finds its one problem before anything is built.

Docker Desktop on Windows

As Docker's own pages state them

Docker Desktop on WSL 2: WSL 2.1.5 or later, 8 GB RAM (docs.docker.com/desktop/setup/install/windows-install, read 2026-10-12).

Docker Personal: US$0 a month, Docker Desktop included (docker.com/pricing, read 2026-10-12).

Docker Desktop without a paid plan: under 250 employees and US$10 million revenue (docker.com/pricing/faq, read 2026-10-12).

Docker Hub: 10 pulls an hour per IP address without a login, 100 an hour with one (docker.com/pricing/faq, read 2026-10-12).

The course does not decide whether the terms cover you. Read them, and follow your firm's policy.

Build, run, request, look inside

```
> docker build -t pfi-bond-service .
[1/6] FROM docker.io/library/python:3.13.16-slim
[2/6] WORKDIR /app
[3/6] COPY requirements.txt .
[4/6] RUN python -m pip install --no-cache-dir -r requirements.txt
[5/6] COPY app.py bondmath.py deploykit.py ./
[6/6] RUN useradd --create-home appuser
```

14

packages installed in the image

12

at the course's versions; pydantic differs

build context: 92.14kB, no .env

```
course image: /app
app.py
bondmath.py
deploykit.py
requirements.txt
```

```
COPY . . image: /app
.env
Dockerfile
app.py
...
```

DV01 per 100 of face

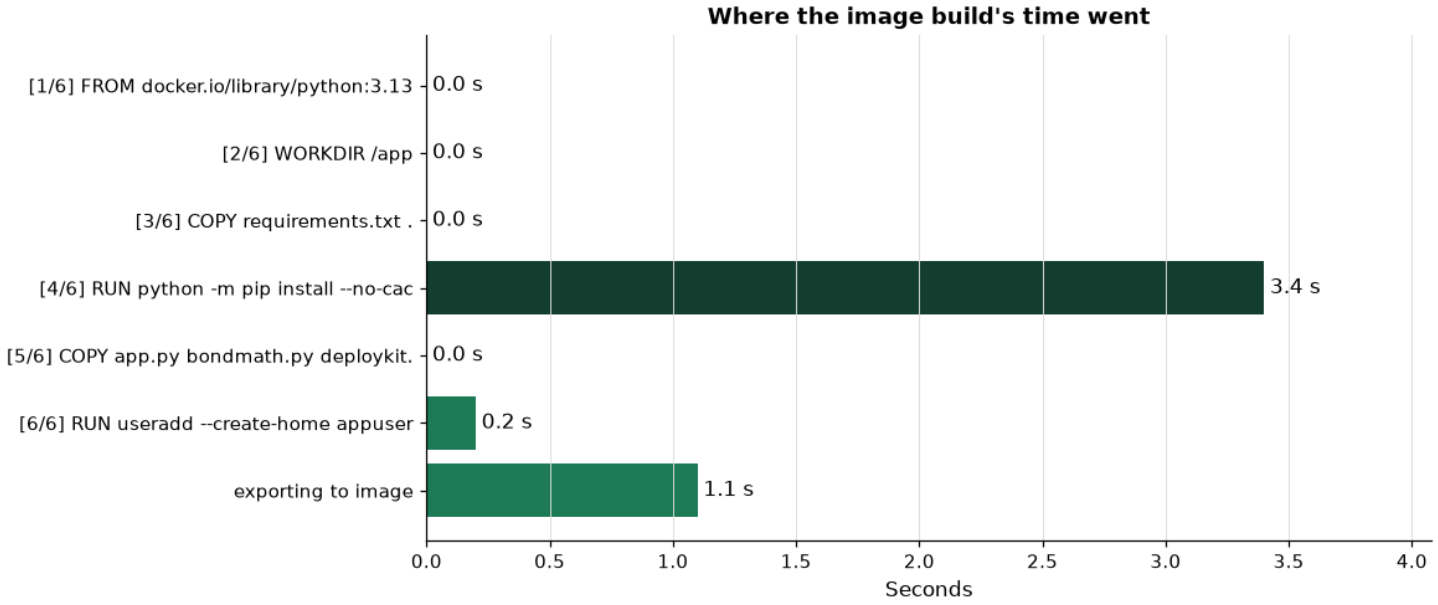
container: 0.04362487919873952

bondmath: 0.04362487919873952

the same to the last digit

Recorded on the course's build machine, masked. Your ids and timings will differ.

Where the build's time went



Step	Seconds
[1/6] FROM docker.io/library	0.0
[2/6] WORKDIR /app	0.0
[3/6] COPY requirements.txt	0.0
[4/6] RUN python -m pip inst	3.4
[5/6] COPY app.py bondmath.p	0.0
[6/6] RUN useradd --create-h	0.2
exporting to image	1.1

One build on one machine, run with --no-cache: the install step is the long one. Yours will differ.

The three images by size

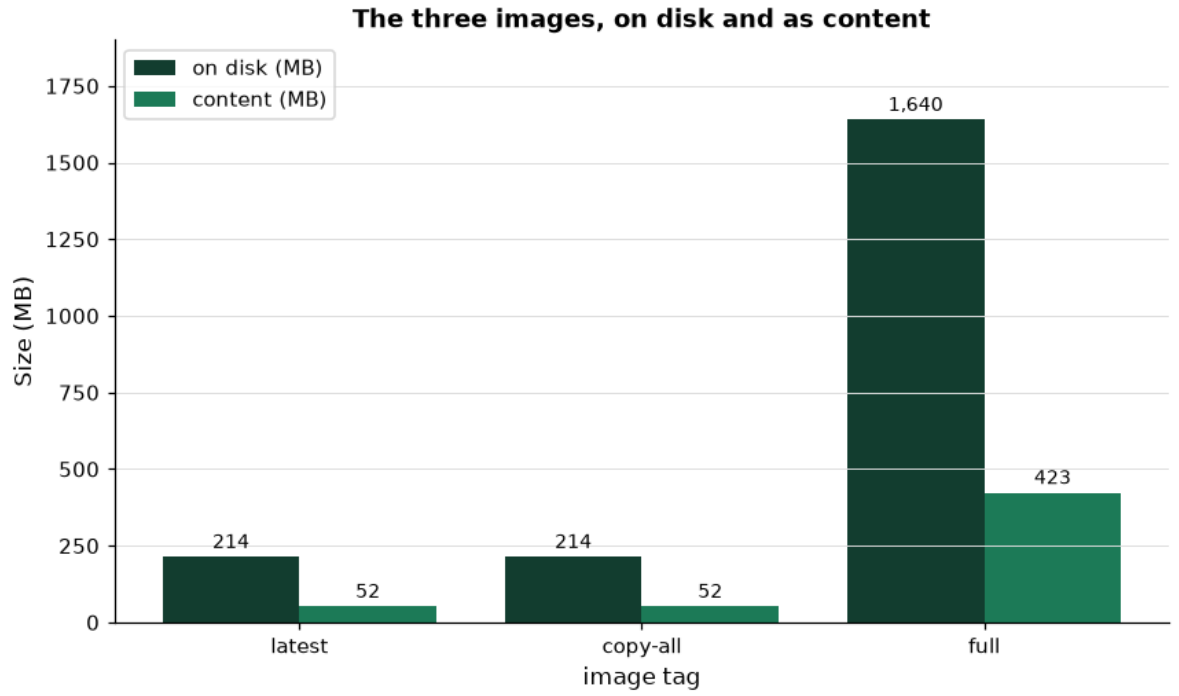
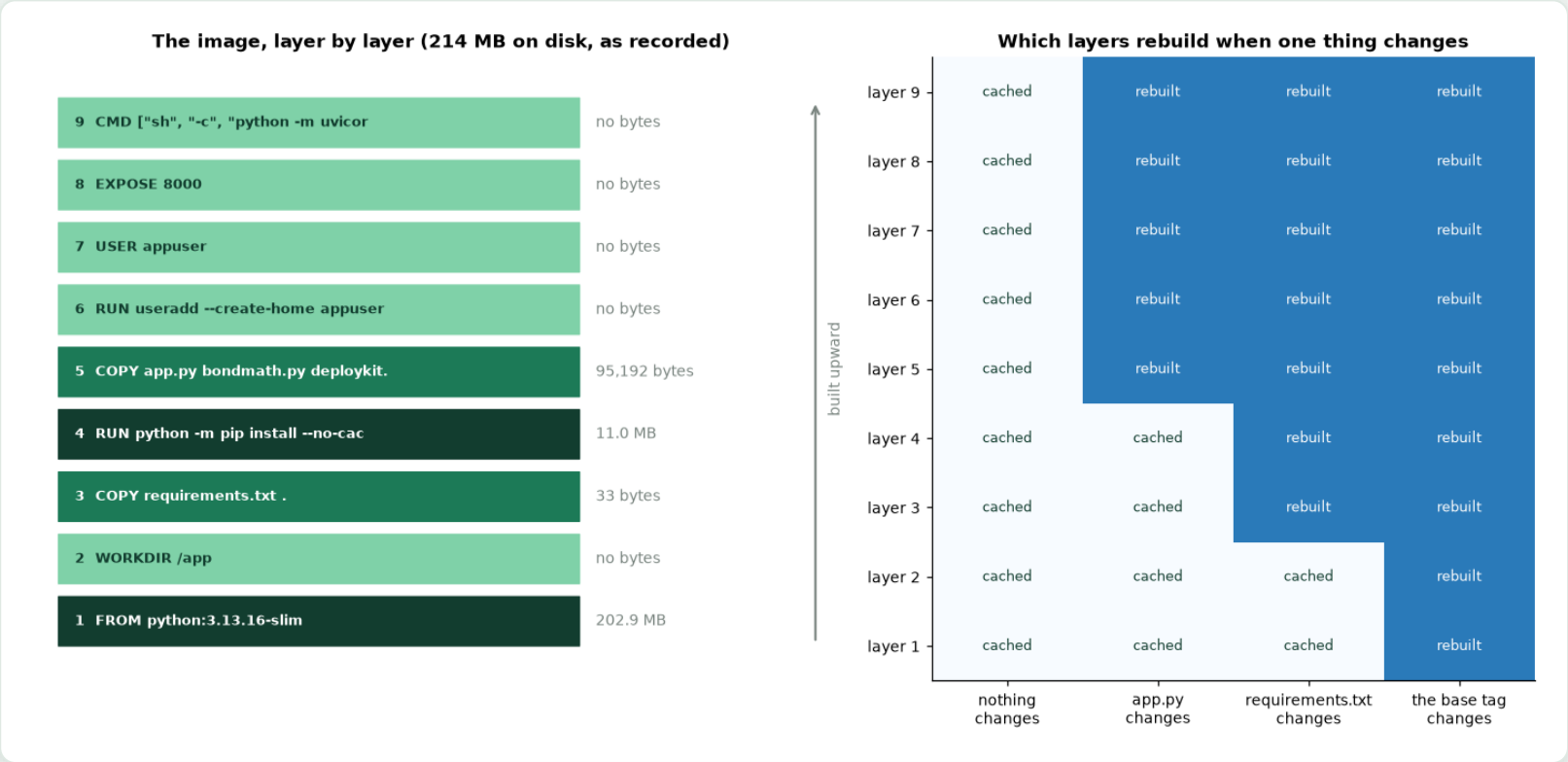


Image	On disk (MB)	Content (MB)
latest	214	52
copy-all	214	52
full	1,640	423

A .env weighs nothing: latest and copy-all are the same size. The full base image is several times larger.

The image, layer by layer



Layer	Adds
1	Debian Linux and Python 3.13.1
2	a folder setting
3	one file
4	FastAPI, uvicorn, and what the
5	three files
6	a user
7	a setting
8	a setting
9	a setting

From the first changed line down, every layer rebuilds: app.py changes 5 layers, requirements.txt 7, the base tag all 9.

Where an image could go public

```
IMAGE   ID   DISK USAGE   CONTENT SIZE   EXTRA
pfi-bond-service:copy-all   bdfef1102da8   214MB   52MB
pfi-bond-service:full   ae8ad0507eba   1.64GB   423MB
pfi-bond-service:latest   0eca51c68053   214MB   52MB   U
```

Render's free web service, as read today

Render Free: spins down after 15 minutes without traffic, about a minute to wake (render.com/docs/free, read 2026-10-12).

Render Free: 750 instance hours a workspace each calendar month (render.com/docs/free, read 2026-10-12).

Render: PORT defaults to 10000; bind to host 0.0.0.0 (render.com/docs/web-services, read 2026-10-12).

Hugging Face: Docker Spaces need a paid plan, PRO for a person (huggingface.co/docs/hub/spaces-overview, read 2026-10-12).

Render: "Do not use them for production applications." The course's image was never pushed.

One image everywhere

PyCon US 2013: a lightning talk

"The future of Linux Containers", Solomon Hykes, Friday 15 March 2013

The record's summary ends: "Until now: Docker."

Source: pyvideo.org/pycon-us-2013/the-future-of-linux-containers.html, read 2026-10-12

Section 11.6, in one line: the same image, the same DV01, wherever it runs.

Excel and Git

Excel side by side: none today

No container, no image. Write down every version.

One commit

modules, app.py, requirements.txt,

Dockerfile, .dockerignore

```
git check-ignore -v .env
.gitignore:3:.env   .env
```

.gitignore keeps .env out of Git; .dockerignore keeps it out of the image. Both must name it.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: COPY . . "so no file is ever left out", and no .dockerignore: the image holds the .env.

Code that runs is not the same as code that is right.

What you can do now

- Say what an image and a container are
- Write a Dockerfile and read every line
- Keep .env out, and prove it with ls -a
- check_dockerfile: five problems that build
- Build, run, request, and stop the service

Week 3, Day 2 Request logs and monitoring